

Bandit - Level 15

Goal

The password for the next level is retrieved by submitting the password of the current level to **port 30001** on localhost, using **SSL/TLS encryption**.

Connection

```
ssh -p 2220 bandit15@bandit.labs.overthewire.org
```

Use the password retrieved at the end of [Bandit - Level 14](#).

Concepts

Same exchange as the previous level, one difference: the channel is encrypted. That difference is the whole point.

What TLS adds

TLS (Transport Layer Security, the successor to SSL) wraps a plain TCP connection in three guarantees:

Guarantee	What it prevents
Confidentiality	anyone in the path reading the traffic
Integrity	anyone modifying it undetected
Authentication	connecting to an impostor server

It's the difference between HTTP and HTTPS, between SMTP and SMTPS, between `nc` and `openssl s_client`. Same data, same ports-and-sockets machinery, a negotiated encryption layer in between.

The handshake

Before a single byte of data moves, client and server run a **handshake**:

1. Client announces the TLS versions and cipher suites it supports
2. Server picks one and sends its **certificate**
3. Client validates that certificate against its trusted CA store
4. Both derive a shared session key using asymmetric cryptography
5. From here on, everything is encrypted with that symmetric key

Step 5 is why TLS is fast despite using public-key crypto: the expensive asymmetric operation happens **once**, only to agree on a symmetric key. Bulk traffic uses the cheap algorithm.

This normally happens invisibly. `s_client` dumps the whole thing on screen, which is what makes it a teaching tool as much as a client.

Why plain `nc` fails against a TLS port

Sending raw text to a TLS listener produces **silence, then a closed connection**. Not an error message — the server received bytes that weren't a valid handshake, couldn't parse them, and hung up.

Absence of a response is itself the diagnosis: a service that accepts the connection and then closes without replying is almost always speaking a protocol the client isn't.

openssl is a multi-tool

openssl isn't one command — it's an umbrella over dozens of subcommands. The **first argument must say what to do**:

```
openssl s_client -connect host:port    # TLS client
openssl x509 -in cert.pem -text        # inspect a certificate
openssl enc -aes-256-cbc -in f -out f.enc  # symmetric encryption
openssl genrsa -out key.pem 4096       # generate a key pair
openssl dgst -sha256 file              # hash a file
openssl base64 -d                      # encode/decode - see [[Linux - Encoding vs Encryption]]
```

Note the syntax mismatch worth remembering: `s_client` wants `host:port` joined by a colon, while `nc` wants them separated by a space. `nc host:port` fails with `missing port number`. **No shared convention exists** — same lesson as the flag letters in [Checkpoint - Niveles 08 a 12](#).

Attempts

```
ls -la ~
# → -rw-r-----  1 bandit15 bandit15  33 .bandit14.password
# A hidden file holding the PREVIOUS level's password. Only visible with -la.
# Convenient, not required - the current password is what this level needs.

cd ~ /etc          # → -bash: cd: too many arguments
cd ~etc/           # → -bash: cd: ~etc/: No such file or directory
```

`~` expands to the home directory only when **alone or followed by a slash**. With a space it becomes two arguments; attached to a word, bash reads `~name` as *that user's* home directory, and no user named `etc` exists.

```
cat /etc/bandit_pass/bandit15
# → pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7

nc 127.0.0.1 30000
pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
# → Wrong! Please enter the correct current password.
# Wrong PORT, not wrong password. 30000 is the previous level's service,
# still listening and still expecting bandit14's password.

nc 127.0.0.1 30001
pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
# → (silence, connection closes)

telnet localhost 30001
```

```
# → Connected to localhost.
# pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
# Connection closed by foreign host.
```

Two different plaintext clients, same outcome: the connection **opens** and then closes with no reply. That confirms the port is live and rules out a firewall — the problem is the protocol, not reachability.

```
nc 127.0.0.1:30001
# → nc: missing port number      (nc wants a space, not a colon)

openssl localhost 30001
# → Invalid command 'localhost'; type "help" for a list.
# openssl needed a SUBCOMMAND first, not a destination.
```

Solution

```
openssl s_client -connect localhost:30001
```

The terminal fills with handshake output, then waits for input exactly like `nc` :

```
pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
# → Correct!
# kS0Hf0u5HiXFwKMKFqXvPd0TNGGa0X8V
# closed
```

```
# Quieter version, suppressing the certificate dump
openssl s_client -connect localhost:30001 -quiet

# Non-interactive
cat /etc/bandit_pass/bandit15 | openssl s_client -connect localhost:30001 -quiet
```

Older `s_client` versions treat a lone `Ctrl+D` or an empty line as a signal to close the connection, cutting off the reply. If the response gets truncated, `-ign_eof` keeps it open.

Reading the handshake output

The wall of text before the prompt is the entire TLS negotiation, printed. It normally happens invisibly on every HTTPS request, so it's worth decoding once.

Certificate validation

```
depth=0 CN=SnakeOil
verify error:num=18:self-signed certificate
```

`depth=0` means the chain has **one certificate**: the server's own, with nothing above it. A real chain would show `depth=1` (intermediate CA) and `depth=2` (root CA).

Error 18 is the self-signed case. **SnakeOil** is the traditional Debian placeholder name for a throwaway test certificate — the term comes from "snake oil" as in fake medicine, a long-standing joke about cryptography of unverifiable quality.

```
0 s:CN=SnakeOil
  i:CN=SnakeOil
```

s: is **subject** (who the certificate is for), **i:** is **issuer** (who signed it). **Identical values mean self-signed:** the certificate vouches for itself, which proves nothing. That single line is the whole diagnosis.

```
a:PKKEY: RSA, 4096 (bit); sigalg: sha256WithRSAEncryption
v:NotBefore: Jun 10 03:59:50 2024 GMT; NotAfter: Jun 8 03:59:50 2034 GMT
```

A 4096-bit RSA key, signed with SHA-256, valid for ten years. Ten years is unusually long — public CAs now cap certificates at ~13 months, precisely so that a compromised key has a limited window.

```
-----BEGIN CERTIFICATE-----
MIIFBzCCAU+gAwIBAgIUBLz7DBxA0IfoljaL/WaJzE6Sbz7cwDQYJKoZIhvcNAQEL
...
-----END CERTIFICATE-----
```

That block is the certificate itself in **PEM** format: **base64 of the binary DER encoding**, wrapped in markers. Same encoding from [Linux - Encoding vs Encryption](#) — and the same point applies: base64 protects nothing. A certificate is *meant* to be public. Inspect it with:

```
openssl x509 -in cert.pem -text -noout
```

Negotiated parameters

```
Negotiated TLS1.3 group: X25519MLKEM768
```

The most interesting line in the output. This is the **key exchange** group, and it's a hybrid:

- **X25519** — elliptic-curve Diffie-Hellman, the current classical standard
- **MLKEM768** — ML-KEM (formerly Kyber), the NIST-standardised **post-quantum** key encapsulation mechanism

Combining both means the session stays secure if *either* holds. The threat model is **harvest now, decrypt later**: an adversary records encrypted traffic today and decrypts it once quantum computers can break elliptic curves. For data with a long secrecy lifetime, that's a present-tense risk, which is why hybrid key exchange rolled out across browsers and servers through 2024–2025.

Seeing it on a wargame server is a good marker of how fast this became default.

```
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
```

The **cipher suite**, decoded:

Part	Meaning
TLS	the protocol
AES_256	symmetric cipher, 256-bit key — encrypts the actual data
GCM	Galois/Counter Mode — provides confidentiality and integrity in one pass
SHA384	hash used for key derivation within the handshake

TLS 1.3 suites are short because the protocol removed all the negotiable weak options that made TLS 1.2 suites long and dangerous.

```
SSL handshake has read 3191 bytes and written 1613 bytes
```

Under 5 KB to establish an authenticated encrypted channel. That's the fixed cost TLS adds per connection — negligible for anything but the highest-volume services, and the reason "HTTPS is slow" stopped being a real argument years ago.

```
Compression: NONE
```

Deliberate, not a limitation. TLS-level compression was removed because it enabled the **CRIME** attack: compressing attacker-controlled data together with a secret leaks the secret through the resulting size. Compression happens at the application layer now, where it can be scoped safely.

```
Post-Handshake New Session Ticket arrived:
  TLS session ticket lifetime hint: 300 (seconds)
  Resumption PSK: 1D2399F7CD8F6E98...
```

A **session ticket** lets a returning client skip the expensive part of the handshake for the next 300 seconds. The server hands over an encrypted blob it can later decrypt to recover the session state — so it stores nothing itself.

```
read R BLOCK
```

Not an error. `s_client` is **blocked waiting for data**, either from the socket or from your keyboard. This is where the password gets typed.

```
Verify return code: 18 (self-signed certificate)
```

And the summary: **confidentiality worked, authentication didn't**. The traffic is encrypted with a 256-bit key, and there is no proof of who is on the other end. A browser would refuse to continue here; `s_client` reports it and carries on, which is exactly what makes it a diagnostic tool rather than a safe client.

That split is the most useful thing in this whole output: **encrypted and trustworthy are two different properties**, and TLS provides them through separate mechanisms that can fail independently.

Points of Friction

1. **Tried port 30000 first.** Returned `Wrong! Please enter the correct current password.` — which reads like a rejected password but was a **wrong port**. The previous level's service is still running and still wants bandit14's credentials. A response that sounds like an answer to your question, but answers a different one: the recurring theme of [Checkpoint - Niveles 08 a 12](#).

2. **Read the silence on 30001 as failure.** It was the answer. A connection that opens and closes without replying means the client is speaking the wrong protocol — the strongest possible hint toward TLS, and it was already in the level's command list (`openssl` , `s_client`).

3. `openssl localhost 30001` . Treated `openssl` as a single command. The error named the problem precisely: `Invalid command 'localhost'; type "help" for a list` . Running `openssl help` would have listed `s_client` immediately.

4. `cd ~ /etc` and `cd ~etc/` . Tilde expansion has narrow rules: alone, or followed by `/` . Everything else means something different or nothing at all.

Key Takeaway

The difference between `nc` and `openssl s_client` is the difference between HTTP and HTTPS.

Identical exchange, identical service shape — one sends the password as readable bytes across the wire, the other negotiates a session key first and sends ciphertext.

Doing Levels 14 and 15 back to back is the clearest possible demonstration of what TLS actually adds, because everything else is held constant.

The diagnostic habit is the second takeaway: **silence is data**. A refused connection means nothing is listening. A connection that opens and immediately closes means something *is* listening and doesn't understand you. Those two failures look similar at a glance and point in completely different directions.

Third, and smaller: `openssl` is worth knowing as a toolbox rather than a command. Inspecting certificates, generating keys, hashing files and testing TLS endpoints all live behind subcommands of the same binary — and it's installed on essentially every Unix system.

Level 15 closes the Month 1 Bandit milestone. Levels 0→15 complete. See [Checkpoint - Niveles 08 a 12](#) for the consolidated review of 8→12; this level and the two before it (SSH keys, netcat, TLS) form the networking block.

Next

```
ssh -p 2220 bandit16@bandit.labs.overthewire.org
```